

Earth System Science – Python Practicals

Practice #1 – Earthquakes on a world map (Ch. 2)

```

1  # =====
2  # Practice #1 : Plot past 30-day earthquake data on a world map
3  # =====
4  # Approach:
5  #   1) Download the CSV file from the USGS feed (or read it locally).
6  #   2) Read it with pandas because it already knows how to parse CSV.
7  #   3) Draw a world map with cartopy and put a red dot for every quake.
8  #   4) Make the dot size depend on the magnitude so big quakes stand out.
9  # -----
10
11 import pandas as pd                # for reading the CSV file
12 import matplotlib.pyplot as plt    # for the plotting itself
13 import cartopy.crs as ccrs         # gives us map projections
14 import cartopy.feature as cfeature # coastlines, borders, etc.
15
16 # URL of the USGS "all earthquakes past 30 days" CSV feed
17 url = "https://earthquake.usgs.gov/earthquakes/feed/v1.0/summary/all_month.csv"
18
19 # read_csv can take a URL directly, no need to download first
20 data = pd.read_csv(url)
21
22 # We only really need the position and the magnitude columns
23 lat = data["latitude"]             # degrees North
24 lon = data["longitude"]            # degrees East
25 mag = data["mag"]                  # Richter-like magnitude
26
27 # ----- Making the figure -----
28 # plt.figure just creates an empty canvas of the chosen size (inches)
29 fig = plt.figure(figsize=(12, 6))
30
31 # add_subplot with a cartopy projection turns the axes into a map;
32 # PlateCarree is the simplest equirectangular projection
33 ax = fig.add_subplot(1, 1, 1, projection=ccrs.PlateCarree())
34
35 # draw the world features so the dots have context
36 ax.add_feature(cfeature.LAND, facecolor="lightgray")
37 ax.add_feature(cfeature.OCEAN, facecolor="lightblue")
38 ax.coastlines(linewidth=0.5)
39
40 # scatter puts one dot per (lon, lat) pair.
41 # s = mag**2 * 3 : make the size grow quickly with magnitude so big
42 # quakes look bigger; transform= tells cartopy the coords are lon/lat.
43 ax.scatter(lon, lat,
44            s=mag**2 * 3,
45            c="red", alpha=0.5, edgecolor="k", linewidth=0.3,
46            transform=ccrs.PlateCarree())
47
48 # gridlines with labels help read the map
49 ax.gridlines(draw_labels=True, linewidth=0.3, color="gray")
50
51 # plt.title just adds the title text on top of the axes
52 plt.title("Earthquakes in the past 30 days (USGS feed)")
53
54 # plt.show pops the figure on screen (or renders it in the notebook)

```

```
55 plt.show()
```

Practice #2 – Vertical pressure profile (Ch. 3)

```

1  # =====
2  # Practice #2 : Vertical pressure profile  $p(z) = p_0 * \exp(-z/H)$ 
3  # =====
4  # Approach:
5  #   - Build the height array z with numpy.
6  #   - Compute analytical pressure and density.
7  #   - Approximate the density using forward / backward / central
8  #     finite differences of p (because  $\rho = -(1/g) dp/dz$ ).
9  #   - Redo with dz=500 and compare with the dz=1000 result.
10 # -----
11
12 import numpy as np
13 import matplotlib.pyplot as plt
14
15 # ----- constants and parameters -----
16 H = 8000.0          # scale height in metres
17 p0 = 1000.0         # surface pressure in hPa
18 g = 9.81            # gravity
19 Rd = 287.0          # gas constant for dry air
20
21 # ----- first case: dz = 1000 m -----
22 dz = 1000.0
23 z = np.arange(0., 9001., dz)  # 0, 1000, ..., 9000 m
24
25 # analytical pressure using the given exponential formula
26 p = p0 * np.exp(-z / H)
27
28 # analytical density :  $\rho(z) = \rho_0 * \exp(-z/H)$ 
29 # We take rho from p using  $p = \rho * Rd * T$  with  $T = 288K$  (rough).
30 T = 288.0
31 rho = (p * 100.0) / (Rd * T)  # multiply by 100 to convert hPa -> Pa
32
33 # ----- numerical derivatives of p -----
34 # forward : ( p[i+1] - p[i] ) / dz
35 # backward: ( p[i] - p[i-1] ) / dz
36 # central : ( p[i+1] - p[i-1] ) / (2 dz)
37 dp_fwd = (p[1:] - p[:-1]) / dz
38 dp_bwd = (p[1:] - p[:-1]) / dz
39 dp_ctr = (p[2:] - p[:-2]) / (2.0 * dz)
40
41 # convert dp/dz into "numerical density" using  $dp/dz = -\rho * g$ 
42 # (Careful: p is in hPa so multiply by 100 first.)
43 rho_fwd = -dp_fwd * 100.0 / g
44 rho_bwd = -dp_bwd * 100.0 / g
45 rho_ctr = -dp_ctr * 100.0 / g
46
47 # ----- plots -----
48 # figure #1 : the pressure profile p(z)
49 plt.figure(figsize=(6,5))
50 # plt.plot draws p on the x-axis and z on the y-axis so height goes up
51 plt.plot(p, z, marker="o", color="blue", label="p(z)")
52 plt.xlabel("Pressure (hPa)")
53 plt.ylabel("Height z (m)")
54 plt.title("Vertical profile of pressure")
55 plt.grid(True)
56 plt.legend()
57
58 # figure #2 : analytical vs numerical density
59 plt.figure(figsize=(6,5))
60 plt.plot(rho, z, "k-", label="analytical rho")
61 plt.plot(rho_fwd, z[:-1], "r--", label="forward diff")
62 plt.plot(rho_bwd, z[1:], "g:", label="backward diff")

```

```

63 plt.plot(rho_ctr, z[1:-1], "b-.", label="central diff")
64 # plt.legend just puts the little colour box at the corner
65 plt.legend()
66 plt.xlabel("Density (kg/m^3)")
67 plt.ylabel("Height (m)")
68 plt.title("Analytical vs numerical density (dz=1000 m)")
69 plt.grid(True)
70
71 # ----- second case: dz = 500 m -----
72 dz2 = 500.0
73 z2 = np.arange(0., 9001., dz2)
74 p2 = p0 * np.exp(-z2 / H)
75 dp2c = (p2[2:] - p2[:-2]) / (2.0 * dz2)
76 rho2 = -dp2c * 100.0 / g
77
78 # figure #3 : comparison of central-difference density at 2 resolutions
79 plt.figure(figsize=(6,5))
80 plt.plot(rho_ctr, z[1:-1], "b-o", label="dz = 1000 m")
81 plt.plot(rho2, z2[1:-1], "r-s", label="dz = 500 m")
82 plt.xlabel("Density (kg/m^3)")
83 plt.ylabel("Height (m)")
84 plt.title("Effect of dz on the central-difference approximation")
85 plt.legend()
86 plt.grid(True)
87
88 plt.show()
89
90 # Conclusion (in the report):
91 # - finer dz (500 m) is closer to the analytical curve
92 # - central difference is the most accurate of the 3 schemes

```

Practice #3.1 – Planck curves (Ch. 4)

```

1  # =====
2  # Practice #3.1 : Planck curves for T = 6000, 5000, 4000, 3000 K
3  # =====
4  # Approach:
5  #   - Define the Planck function B_lambda(T) with c1 = 3.74e-16
6  #     and c2 = 1.45e-2 (from the lecture).
7  #   - Loop over the 4 temperatures, compute B, plot it.
8  #   - Peak wavelength = wavelength where B is maximum (numpy.argmax).
9  #   - Total emitted energy per unit area = sigma * T^4 (Stefan-Boltzmann);
10 #     we also cross-check with a trapezoidal integration of B.
11 # -----
12
13 import numpy as np
14 import matplotlib.pyplot as plt
15
16 # constants
17 c1 = 3.74e-16      # W m^-2
18 c2 = 1.45e-2       # m K
19 sigma = 5.67e-8    # Stefan-Boltzmann
20
21 # wavelength grid: from 0.01 to 3 micrometres, expressed in metres
22 # (we avoid 0 to prevent division by zero in the exponential)
23 lam = np.linspace(0.01e-6, 3.0e-6, 2000)
24
25 # temperatures we have to plot
26 temperatures = [6000, 5000, 4000, 3000]
27
28 plt.figure(figsize=(8,5))
29
30 for T in temperatures:
31     # Planck's formula in SI units.
32     B = (c1 * lam**-5) / (np.pi * (np.exp(c2 / (lam * T)) - 1.0))
33

```

```

34 # plot in micrometres for readability (multiply lam by 1e6)
35 plt.plot(lam * 1e6, B, label="T = " + str(T) + " K")
36
37 # peak wavelength: take argmax to find the index of the maximum
38 i_peak = np.argmax(B)
39 lam_peak_um = lam[i_peak] * 1e6
40 print("T =", T, "K -> peak wavelength ~", round(lam_peak_um,3), "um")
41
42 # total energy: Stefan-Boltzmann
43 E_sb = sigma * T**4
44 # numerical integral of B over all wavelengths (times pi steradians)
45 E_num = np.trapz(B, lam) * np.pi
46 print("          Stefan-Boltzmann E =", E_sb, " ; numerical ~", E_num)
47
48 plt.xlabel("Wavelength (micrometre)")
49 plt.ylabel("B_lambda (W m^-2 sr^-1 m^-1)")
50 plt.title("Planck curves for different blackbody temperatures")
51 # plt.legend puts the labels ("T=6000 K", etc.) in a little box
52 plt.legend()
53 plt.grid(True)
54 plt.show()

```

Practice #3.2 – 0-D Energy Balance Model (Ch. 4)

```

1 # =====
2 # Practice #3.2 : 0-D EBM -> Ts as a function of eps and alpha
3 # =====
4 # Approach:
5 # The energy-balance equation at the top of atmosphere gives:
6 #       $S_0/4 * (1 - \alpha) = \epsilon \sigma T_s^4$ 
7 # -> so  $T_s = (S_0 * (1 - \alpha) / (4 * \epsilon * \sigma))^{0.25}$ 
8 #
9 # Part 1: fix alpha=0.32, vary eps in (0, 1] -> plot Ts(eps)
10 # Part 2: fix eps=0.66, vary alpha in [0, 1) -> plot Ts(alpha)
11 # Part 3 (bonus): make a 2-D surface Ts(alpha, eps).
12 # -----
13
14 import numpy as np
15 import matplotlib.pyplot as plt
16
17 S0 = 1368.0
18 sigma = 5.67e-8
19
20 def surface_T(alpha, eps):
21     """Return Ts from the simple 0-D energy balance."""
22     return (S0 * (1.0 - alpha) / (4.0 * eps * sigma)) ** 0.25
23
24 # ----- Part 1 : Ts(eps) at alpha = 0.32 -----
25 alpha_fix = 0.32
26 eps_arr = np.linspace(0.01, 1.0, 100) # start at 0.01 (avoid /0)
27 Ts_eps = surface_T(alpha_fix, eps_arr)
28
29 plt.figure(figsize=(6,4))
30 plt.plot(eps_arr, Ts_eps, "r-")
31 plt.xlabel("Emissivity eps")
32 plt.ylabel("Surface temperature Ts (K)")
33 plt.title("Ts as function of eps (alpha = 0.32)")
34 plt.grid(True)
35
36 # ----- Part 2 : Ts(alpha) at eps = 0.66 -----
37 eps_fix = 0.66
38 alpha_arr = np.linspace(0.0, 1.0 - 1e-3, 100) # avoid alpha=1
39 Ts_alpha = surface_T(alpha_arr, eps_fix)
40
41 plt.figure(figsize=(6,4))
42 plt.plot(alpha_arr, Ts_alpha, "b-")

```

```

43 plt.xlabel("Planetary albedo alpha")
44 plt.ylabel("Surface temperature Ts (K)")
45 plt.title("Ts as function of alpha (eps = 0.66)")
46 plt.grid(True)
47
48 # ----- Part 3 : 2-D map Ts(alpha, eps) -----
49 # np.meshgrid turns the two 1D arrays into a rectangular grid
50 # so we can evaluate the function on the entire (alpha, eps) plane.
51 A, E = np.meshgrid(alpha_arr, eps_arr)
52 Ts2d = surface_T(A, E)
53
54 plt.figure(figsize=(7,5))
55 # contourf makes a filled contour plot (nicer than pcolormesh for math)
56 cf = plt.contourf(A, E, Ts2d, 20, cmap="viridis")
57 # plt.colorbar attaches the colour scale on the right of the figure
58 cbar = plt.colorbar(cf)
59 cbar.set_label("Ts (K)")
60 plt.xlabel("alpha")
61 plt.ylabel("eps")
62 plt.title("Surface temperature Ts vs alpha and eps")
63
64 plt.show()

```

Practice #4.1 – Greenhouse effect (Ch. 5)

```

1  # =====
2  # Practice #4.1 : 1-layer greenhouse model
3  # =====
4  # Approach:
5  #   The atmosphere absorbs a fraction eps of the outgoing IR from
6  #   the ground and re-emits eps*sigma*Ta^4 both upward and downward.
7  #   Solving the energy balances at TOA and at the surface gives:
8  #       Ts = ( S0*(1-alpha)/(sigma*(4-2*eps)) ) ** 0.25
9  #       Ta = ( eps * Ts^4 / 2 ) ** 0.25
10 #   We just loop eps from ~0 to 1 in steps of 0.01 and plot.
11 # -----
12
13 import numpy as np
14 import matplotlib.pyplot as plt
15
16 S0      = 1361.0
17 alpha_g = 0.3
18 sigma   = 5.67e-8
19
20 # np.arange(start, stop, step) generates 0.01, 0.02, ...
21 eps_vals = np.arange(0.01, 1.001, 0.01) # start >0 to keep formulas safe
22
23 Ts_list = [] # empty lists that we fill inside the loop
24 Ta_list = []
25
26 for eps in eps_vals:
27     # Ts from the algebra above
28     Ts = ( S0 * (1 - alpha_g) / (sigma * (4 - 2*eps)) ) ** 0.25
29     # Ta from radiative balance of the atmosphere alone
30     Ta = ( eps * Ts**4 / 2.0 ) ** 0.25
31     Ts_list.append(Ts)
32     Ta_list.append(Ta)
33
34 # convert to numpy arrays so plt.plot is happy
35 Ts_arr = np.array(Ts_list)
36 Ta_arr = np.array(Ta_list)
37
38 plt.figure(figsize=(7,5))
39 # two curves on the same axes
40 plt.plot(eps_vals, Ts_arr, "r-", label="Surface Ts")
41 plt.plot(eps_vals, Ta_arr, "b--", label="Atmosphere Ta")

```

```

42 plt.xlabel("Emissivity eps")
43 plt.ylabel("Temperature (K)")
44 plt.title("1-layer greenhouse model")
45 plt.legend()
46 plt.grid(True)
47 plt.show()
48
49 # Comment (report):
50 # - When eps -> 0, no atmosphere: Ts is the naked planet ~ 255 K
51 # - When eps -> 1, strong greenhouse: Ts rises above 300 K

```

Practice #4.2 – 8-layer vertical temperature (Ch. 5)

```

1  # =====
2  # Practice #4.2 : n=8 layers grey atmosphere
3  # =====
4  # Approach:
5  #   For a grey atmosphere fully absorbing IR (eps=1) with n identical
6  #   layers, one can show
7  #        $T_{\text{layer}_k}^4 = (n - k + 1) * T_e^4$    ( $k = 1..n$ ,  $k=1$  is top)
8  #        $T_{\text{surface}}^4 = (n + 1) * T_e^4$ 
9  #   where  $T_e = ((S_0 * (1 - \alpha_g)) / (4 * \sigma))^{0.25}$  is the effective T.
10 #   We compute  $T_e$  then loop over the 8 layers.
11 # -----
12
13 import numpy as np
14 import matplotlib.pyplot as plt
15
16 S0      = 1361.0
17 alpha_g = 0.3
18 sigma   = 5.67e-8
19 n       = 8
20
21 # emission temperature (a.k.a. effective temperature)
22 Te = ( S0 * (1 - alpha_g) / (4 * sigma) ) ** 0.25
23 print("Effective temperature Te =", round(Te, 2), "K")
24
25 # temperatures of each layer, from top (k=1) to bottom (k=n)
26 layer_T = []
27 for k in range(1, n + 1):
28     Tk = ( (n - k + 1) * Te**4 ) ** 0.25
29     layer_T.append(Tk)
30
31 # surface temperature
32 Ts = ( (n + 1) * Te**4 ) ** 0.25
33 print("Surface temperature Ts =", round(Ts, 2), "K")
34
35 # relative height: assume equal thickness, so layer k sits at
36 # a normalised height of 1 - (k - 0.5)/n. Surface is at 0.
37 heights = [1 - (k - 0.5)/n for k in range(1, n + 1)]
38
39 # add the surface point at height 0 for the plot
40 all_T = [Ts] + layer_T
41 all_z = [0.0] + heights
42
43 plt.figure(figsize=(6,6))
44 # plot T on x-axis, height on y-axis (looks like an atmospheric sounding)
45 plt.plot(all_T, all_z, "o-", color="darkred")
46 plt.xlabel("Temperature (K)")
47 plt.ylabel("Relative height (0 = surface, 1 = top)")
48 plt.title("Vertical temperature profile in an 8-layer atmosphere")
49 plt.grid(True)
50 plt.show()
51
52 # Comment: T decreases with height as expected; the surface is the
53 # hottest because it receives back-radiation from all 8 layers.

```

Practice #6 – Daisyworld (Ch. 6)

```

1  # =====
2  # Practice #6 : Daisyworld with black and white daisies
3  # =====
4  # Approach:
5  # 1) Plot the growth rate  $f = \max(0, 1 - k \cdot (T_0 - T)^2)$  vs. temperature.
6  # 2) At  $S = S_0 = 1000$ , start with  $b = w = 0.2$  and iterate the model
7  #    until it converges (differences below a small tolerance).
8  # 3) Sweep  $S/S_0$  from 0.4 to 1.6, find equilibrium at each step,
9  #    then plot equilibrium  $b$ ,  $w$  and planetary  $T$  against  $S/S_0$ .
10 # -----
11
12 import numpy as np
13 import matplotlib.pyplot as plt
14
15 # ----- constants -----
16 k      = 0.003265
17 T0     = 22.5 + 273.15      # ideal temperature in Kelvin
18 gamma  = 0.3               # death rate of daisies
19 sigma  = 5.67e-8
20 S0_ref = 1000.0            # W/m2 (as in the lecture)
21 albedo_b = 0.25            # black daisies
22 albedo_w = 0.75            # white daisies
23 albedo_bare = 0.5
24 q = 20.0                   # local heat transfer factor (K4)
25
26 # ----- part 1: growth rate curves -----
27 T_range = np.linspace(0 + 273.15, 45 + 273.15, 200)
28 #  $\max(0, \dots)$  can't be applied element-wise; use  $\text{np.maximum}$  instead
29 fb_curve = np.maximum(0.0, 1 - k * (T0 - T_range)**2)
30 fw_curve = np.maximum(0.0, 1 - k * (T0 - T_range)**2)      # same formula
31
32 plt.figure(figsize=(6,4))
33 plt.plot(T_range - 273.15, fb_curve, "k-", label="fb (black)")
34 plt.plot(T_range - 273.15, fw_curve, "b--", label="fw (white)")
35 plt.xlabel("Local temperature (deg C)")
36 plt.ylabel("Growth rate")
37 plt.title("Daisy growth rate vs temperature")
38 plt.legend()
39 plt.grid(True)
40
41 # ----- helper: iterate to equilibrium -----
42 def run_daisyworld(S, b_init=0.2, w_init=0.2,
43                   max_iter=5000, tol=1e-6):
44     """Iterate the Daisyworld equations until b, w stop moving."""
45     b = b_init
46     w = w_init
47     for step in range(max_iter):
48         x = 1.0 - b - w                                # bare ground
49         # planetary albedo weighted by cover
50         alpha_p = b*albedo_b + w*albedo_w + x*albedo_bare
51         # planetary temperature from radiative balance
52         Te4 = S * (1 - alpha_p) / sigma
53         Te = Te4 ** 0.25
54         # local patch temperatures
55         Tb = ( q * (alpha_p - albedo_b) + Te4 ) ** 0.25
56         Tw = ( q * (alpha_p - albedo_w) + Te4 ) ** 0.25
57         # growth rates
58         fb = max(0.0, 1 - k * (T0 - Tb)**2)
59         fw = max(0.0, 1 - k * (T0 - Tw)**2)
60         # Euler step (dt = 1) : db/dt = b * (x*fb - gamma)
61         db = b * (x * fb - gamma)
62         dw = w * (x * fw - gamma)

```

```

63     b_new = b + db
64     w_new = w + dw
65     # keep a tiny seed so species can never die out completely
66     b_new = max(b_new, 0.01)
67     w_new = max(w_new, 0.01)
68     # convergence test
69     if abs(b_new - b) < tol and abs(w_new - w) < tol:
70         b, w = b_new, w_new
71         break
72     b, w = b_new, w_new
73     return b, w, Te
74
75 # ----- part 2: single run at S = S0 -----
76 b_eq, w_eq, T_eq = run_daisyworld(S0_ref)
77 print("S = S0: b_eq =", round(b_eq,3), " w_eq =", round(w_eq,3),
78       " T =", round(T_eq - 273.15, 2), "degC")
79
80 # ----- part 3: sweep S/S0 -----
81 ratios = np.linspace(0.4, 1.6, 40)
82 b_list, w_list, T_list = [], [], []
83
84 for r in ratios:
85     b_eq, w_eq, T_eq = run_daisyworld(S0_ref * r)
86     b_list.append(b_eq)
87     w_list.append(w_eq)
88     T_list.append(T_eq - 273.15)    # store in degC for plotting
89
90 # equilibrium areas
91 plt.figure(figsize=(7,4))
92 plt.plot(ratios, b_list, "k-o", label="black daisies b")
93 plt.plot(ratios, w_list, "b-s", label="white daisies w")
94 plt.xlabel("S / S0")
95 plt.ylabel("Equilibrium cover fraction")
96 plt.title("Daisy populations vs solar luminosity")
97 plt.legend()
98 plt.grid(True)
99
100 # planetary temperature
101 plt.figure(figsize=(7,4))
102 plt.plot(ratios, T_list, "r-o", label="Planet T (with daisies)")
103 # for comparison, planet without daisies (albedo = bare ground)
104 T_bare = ((S0_ref * ratios * (1 - albedo_bare) / sigma) ** 0.25) - 273.15
105 plt.plot(ratios, T_bare, "k--", label="No daisies")
106 plt.xlabel("S / S0")
107 plt.ylabel("Planetary T (deg C)")
108 plt.title("Regulation of planetary T by Daisyworld")
109 plt.legend()
110 plt.grid(True)
111
112 plt.show()

```

Practice #7.1 – Vertical velocity from a 3x3 grid (Ch. 7)

```

1  # =====
2  # Practice #7.1 : vertical velocity at the centre of a 3x3 grid
3  # =====
4  # Approach:
5  #   For an incompressible atmosphere, the continuity equation is
6  #       du/dx + dv/dy + dw/dz = 0
7  #   Central differences at the central point over the 3x3 grid:
8  #       du/dx = (u_right - u_left) / (2 * dx)
9  #       dv/dy = (v_north - v_south) / (2 * dy)
10 #   Then integrate vertically from z=0 (where w=0) to z=1000 m:
11 #       w(1000) = w(0) - (du/dx + dv/dy) * dz
12 # -----
13

```

```

14 import numpy as np
15
16 # grid spacing (m)
17 dx = 100000.0
18 dy = 100000.0
19 dz = 1000.0
20
21 # a fake 3x3 wind field (row 0 = north row, row 2 = south row)
22 # rows are latitude (y), columns are longitude (x)
23 # Replace by the numbers from the exercise if a specific table is given.
24 u = np.array([[ 5.0,  6.0,  7.0],
25               [ 4.0,  5.0,  6.0],
26               [ 3.0,  4.0,  5.0]]) # m/s , eastward
27
28 v = np.array([[ -2.0, -1.0,  0.0],
29               [ -1.0,  0.0,  1.0],
30               [  0.0,  1.0,  2.0]]) # m/s , northward
31
32 # central differences at the middle point (index [1,1])
33 # note that "top" in matrix terms is row index 0 (usually north)
34 du_dx = (u[1,2] - u[1,0]) / (2.0 * dx)
35 dv_dy = (v[0,1] - v[2,1]) / (2.0 * dy)
36
37 # incompressibility -> dw/dz = -(du/dx + dv/dy)
38 dw_dz = -(du_dx + dv_dy)
39
40 # integrate from z=0 (w=0) up to z=1000 m
41 w0 = 0.0
42 w_1000 = w0 + dw_dz * dz
43
44 print("du/dx =", du_dx, "1/s")
45 print("dv/dy =", dv_dy, "1/s")
46 print("dw/dz =", dw_dz, "1/s")
47 print("Vertical velocity at z=1000 m : w =", w_1000, "m/s")
48 if w_1000 > 0:
49     print("-> Air is RISING at the central station.")
50 else:
51     print("-> Air is SINKING at the central station.")

```

Practice #7.2 – NCEP wind field (Ch. 7)

```

1 # =====
2 # Practice #7.2 : NCEP wind field + vertical velocity map
3 # =====
4 # Approach:
5 # - Open the two remote NetCDF files with xarray (OPeNDAP URLs).
6 # - Pick the requested time (2026-03-01 12:00) and level (850 hPa).
7 # - Compute w on a horizontal grid by integrating the horizontal
8 #   divergence from the surface (w = 0) up to 850 hPa.
9 # - Show wind vectors with quiver and w with contourf.
10 # -----
11
12 import numpy as np
13 import matplotlib.pyplot as plt
14 import xarray as xr
15 import cartopy.crs as ccrs
16 import cartopy.feature as cfeature
17
18 url_u = "https://psl.noaa.gov/thredds/dodsC/Datasets/ncep.reanalysis/pressure/uwnd
19         .2026.nc"
20 url_v = "https://psl.noaa.gov/thredds/dodsC/Datasets/ncep.reanalysis/pressure/vwnd
21         .2026.nc"
22
23 target_time = "2026-03-01T12:00:00"
24 target_level = 850

```

```

24 # open_dataset can take a URL if it is an OPeNDAP endpoint
25 ds_u = xr.open_dataset(url_u)
26 ds_v = xr.open_dataset(url_v)
27
28 # .sel picks the closest matching time / level automatically
29 u = ds_u["uwnd"].sel(time=target_time, level=target_level)
30 v = ds_v["vwnd"].sel(time=target_time, level=target_level)
31
32 lat = u["lat"].values
33 lon = u["lon"].values
34
35 # radius of Earth in metres
36 R = 6.371e6
37
38 # convert dlon, dlat (deg) into physical metres using cos(lat) for lon
39 LAT, LON = np.meshgrid(lat, lon, indexing="ij")
40 dlat = np.deg2rad(np.gradient(lat))
41 dlon = np.deg2rad(np.gradient(lon))
42
43 # horizontal derivatives with numpy.gradient
44 du_dx = np.gradient(u.values, axis=1) / (R * np.cos(np.deg2rad(LAT)) * dlon[np.
    newaxis,:])
45 dv_dy = np.gradient(v.values, axis=0) / (R * dlat[:, np.newaxis])
46
47 # vertical velocity between surface (w = 0) and 850 hPa
48 # approximating dz between surface and 850 hPa as ~1500 m
49 dz = 1500.0
50 w = -(du_dx + dv_dy) * dz # simple layer approximation
51
52 # ----- figure -----
53 fig = plt.figure(figsize=(12,6))
54 ax = fig.add_subplot(1,1,1, projection=ccrs.PlateCarree())
55 ax.coastlines(linewidth=0.5)
56 ax.add_feature(cfeature.BORDERS, linewidth=0.3)
57
58 # contourf : fills the map with the vertical velocity in colour
59 cf = ax.contourf(lon, lat, w, 20, cmap="RdBu_r",
60                 transform=ccrs.PlateCarree())
61 # colorbar : the little scale on the side
62 cbar = plt.colorbar(cf, ax=ax, shrink=0.7)
63 cbar.set_label("w (m/s) (positive = rising)")
64
65 # quiver : draws wind vectors. Downsample so the plot stays readable.
66 step = 5
67 ax.quiver(lon[::step], lat[::step],
68           u.values[::step, ::step], v.values[::step, ::step],
69           transform=ccrs.PlateCarree(),
70           scale=700, width=0.002)
71
72 ax.set_title("Wind at 850 hPa on " + target_time)
73 ax.gridlines(draw_labels=True, linewidth=0.3, color="gray")
74
75 plt.show()

```

Practice #8 – Thermal lag (Ch. 8)

```

1 # =====
2 # Practice #8 : thermal lag between ground and near-surface air
3 # =====
4 # Approach:
5 #   Integrate two coupled ODEs:
6 #        $C_g \frac{dT_g}{dt} = SW_{in} - LW_{out} - QH$ 
7 #        $C_a \frac{dT_a}{dt} = QH$ 
8 #   with  $QH = h * (T_g - T_a)$ ,  $SW_{in}$  a positive sine during the day
9 #   and 0 at night, and  $LW_{out} = \sigma * T_g^4$ .
10 #   Use forward Euler with  $dt = 60$  s and run for 3 days so the

```

```

11 # system forgets its initial condition and shows the diurnal cycle.
12 # -----
13
14 import numpy as np
15 import matplotlib.pyplot as plt
16
17 # ----- parameters -----
18 Cg      = 2.0e5          # ground thermal inertia (J / m2 / K)
19 Ca      = 1.0e4          # air thermal inertia
20 h       = 15.0           # heat transfer coefficient
21 sigma   = 5.67e-8
22 dt      = 60.0           # time step in seconds
23 n_days  = 3
24 n_steps = int(n_days * 24 * 3600 / dt)
25
26 # initial temperatures (K) at t = 0 (midnight)
27 Tg = 18.0 + 273.15
28 Ta = 18.0 + 273.15
29
30 # time in hours, starting from midnight
31 time_hours = np.arange(n_steps) * dt / 3600.0
32
33 # arrays to store the results at every time step
34 Tg_hist = np.zeros(n_steps)
35 Ta_hist = np.zeros(n_steps)
36 SW_hist = np.zeros(n_steps)
37 LW_hist = np.zeros(n_steps)
38
39 for i in range(n_steps):
40     t_hours = time_hours[i]
41     # local time-of-day between 0 and 24
42     hour = t_hours % 24
43     # incoming SW: sine peaking at 12:00 with 800 W/m2, 0 at night
44     if 6 <= hour <= 18:
45         SW = 800.0 * np.sin(np.pi * (hour - 6) / 12.0)
46     else:
47         SW = 0.0
48     # outgoing LW from the ground
49     LW = sigma * Tg**4
50     # sensible heat flux
51     QH = h * (Tg - Ta)
52     # explicit Euler update
53     Tg = Tg + dt * (SW - LW - QH) / Cg
54     Ta = Ta + dt * (QH) / Ca
55
56     Tg_hist[i] = Tg
57     Ta_hist[i] = Ta
58     SW_hist[i] = SW
59     LW_hist[i] = LW
60
61 # only plot the last 48 hours so the initial transient is gone
62 last = time_hours > (n_days - 2) * 24
63 t_plot = time_hours[last] - (n_days - 2) * 24
64
65 fig, ax1 = plt.subplots(figsize=(9,5))
66 # left y-axis : temperature curves
67 ax1.plot(t_plot, Tg_hist[last] - 273.15, "r-", label="Tg (ground)")
68 ax1.plot(t_plot, Ta_hist[last] - 273.15, "b--", label="Ta (air)")
69 ax1.set_xlabel("Local time (h)")
70 ax1.set_ylabel("Temperature (deg C)")
71 ax1.grid(True)
72 ax1.legend(loc="upper left")
73
74 # right y-axis : SW and LW radiation (twinx shares the x-axis)
75 ax2 = ax1.twinx()
76 ax2.plot(t_plot, SW_hist[last], "y:", label="SW in")

```

```

77 ax2.plot(t_plot, LW_hist[last], "k-.", label="LW out")
78 ax2.set_ylabel("Radiation flux (W/m^2)")
79 ax2.legend(loc="upper right")
80
81 # find the time of maximum Tg over the last 24h and print it
82 last24 = t_plot >= 24
83 i_max = np.argmax(Tg_hist[last][last24])
84 tmax = t_plot[last24][i_max]
85 print("Ground Tg is maximum around", round(tmax % 24, 2), "h local time")
86 print("SW is maximum at 12:00 -> the delay is the thermal lag.")
87
88 plt.title("Thermal lag: peak Tg after solar noon")
89 plt.show()

```

Practice #9 – Virtual temperature from ERA5 (Ch. 9)

```

1  # =====
2  # Practice #9 : virtual temperature from ERA5 data
3  # =====
4  # Approach:
5  #   - Download / read the netCDF file with xarray.
6  #   - Virtual temperature:  $T_v = T * (1 + 0.61 * q)$  with  $q$  in kg/kg.
7  #   - Take the zonal mean (average over longitude) to get a
8  #     latitude-vs-pressure cross-section.
9  #   - Plot  $T_v$  (2D) and the difference  $T_v - T$  (2D).
10 # -----
11
12 import numpy as np
13 import matplotlib.pyplot as plt
14 import xarray as xr
15
16 url = "https://remosat.usth.edu.vn/~thanhnhd/Download/tmp/ERA5_SH_Temp.nc"
17 ds = xr.open_dataset(url)
18 # ERA5 conventions: "t" is temperature, "q" is specific humidity
19 T = ds["t"] # K
20 q = ds["q"] # kg/kg
21
22 # virtual temperature; broadcasts naturally over all dimensions
23 Tv = T * (1.0 + 0.61 * q)
24
25 # zonal mean: average along longitude to get 2-D field (lat, level)
26 Tv_zm = Tv.mean(dim="longitude")
27 T_zm = T.mean(dim="longitude")
28 diff = Tv_zm - T_zm
29
30 # ----- figure #1 : Tv (lat vs pressure) -----
31 plt.figure(figsize=(8,5))
32 # contourf fills the area between contour levels with colour
33 cf = plt.contourf(Tv_zm["latitude"], Tv_zm["level"], Tv_zm.values,
34                  20, cmap="RdYlBu_r")
35 plt.gca().invert_yaxis() # pressure decreases with height
36 plt.colorbar(cf, label="Tv (K)")
37 plt.xlabel("Latitude (deg)")
38 plt.ylabel("Pressure level (hPa)")
39 plt.title("Zonal mean virtual temperature")
40
41 # ----- figure #2 : Tv - T -----
42 plt.figure(figsize=(8,5))
43 cf = plt.contourf(diff["latitude"], diff["level"], diff.values,
44                  20, cmap="hot_r")
45 plt.gca().invert_yaxis()
46 plt.colorbar(cf, label="Tv - T (K)")
47 plt.xlabel("Latitude (deg)")
48 plt.ylabel("Pressure level (hPa)")
49 plt.title("Difference Tv - T (moisture correction)")
50

```

```

51 plt.show()
52
53 # Comment: the largest differences show up in the tropics and near
54 # the surface, where  $q$  is largest -> potentially unstable region.

```

Practice #10.1 – Global mean sea level from CSV (Ch. 10)

```

1  # =====
2  # Practice #10.1 : read altimetry CSV and plot sea level rise
3  # =====
4  # Approach:
5  #   - The CSV comes with 4 mission columns (Topex/Poseidon, Jason 1-3).
6  #   - Many entries are NaN because each mission only spans some years.
7  #   - Use numpy.genfromtxt to read and np.isnan to filter NaNs
8  #     mission by mission, then plot each mission with a different colour.
9  # -----
10
11 import numpy as np
12 import matplotlib.pyplot as plt
13
14 fname = "slr_sla_gbl_keep-txj1j2_90.csv" # local file
15 # comments='#' tells genfromtxt to skip header lines starting with '#'
16 data = np.genfromtxt(fname, delimiter=',', comments='#')
17
18 time = data[:, 0] # decimal year
19 topex = data[:, 1]
20 jason1 = data[:, 2]
21 jason2 = data[:, 3]
22 jason3 = data[:, 4]
23
24 # helper : remove NaN values from a series and its time vector
25 def clean(t, series):
26     good = ~np.isnan(series) # ~ inverts True/False mask
27     return t[good], series[good]
28
29 t_tp, tp = clean(time, topex)
30 t_j1, j1 = clean(time, jason1)
31 t_j2, j2 = clean(time, jason2)
32 t_j3, j3 = clean(time, jason3)
33
34 plt.figure(figsize=(9,5))
35 plt.plot(t_tp, tp, lw=2, label="Topex/Poseidon")
36 plt.plot(t_j1, j1, lw=2, label="Jason-1")
37 plt.plot(t_j2, j2, lw=2, label="Jason-2")
38 plt.plot(t_j3, j3, lw=2, label="Jason-3")
39 # legend location "upper left" is where the empty corner of the plot is
40 plt.legend(loc="upper left")
41 plt.xlabel("Year")
42 plt.ylabel("Sea level rise (mm)")
43 plt.title("Global mean sea level")
44 plt.grid(True)
45 plt.show()

```

Practice #10.2 – Sea Surface Temperature (Ch. 10)

```

1  # =====
2  # Practice #10.2 : SST mean and anomaly maps
3  # =====
4  # Approach:
5  #   - Open sst.mnmean.nc with netCDF4.
6  #   - Compute the time mean SST (climatology) over the whole record.
7  #   - Compute the SST departure for a chosen recent month:
8  #     anomaly = sst(that month) - climatology(same calendar month)
9  #   - Draw both fields on a world map with cartopy.
10 # -----

```

```

11
12 import numpy as np
13 import matplotlib.pyplot as plt
14 import netCDF4 as nc
15 import cartopy.crs as ccrs
16
17 # open the file
18 file = "sst.mnmean.nc"
19 fh = nc.Dataset(file, "r")
20
21 lat = fh.variables["lat"][:]
22 lon = fh.variables["lon"][:]
23 sst = fh.variables["sst"][:] # shape = (time, lat, lon)
24 time = fh.variables["time"]
25 dtimes = nc.num2date(time[:], time.units)
26 fh.close()
27
28 # ----- climatological mean over all months -----
29 sst_mean = np.mean(sst, axis=0)
30
31 # ----- departure for the last available month -----
32 last_month = dtimes[-1].month
33 same_month_mask = np.array([d.month == last_month for d in dtimes])
34 climato_same = np.mean(sst[same_month_mask], axis=0)
35 sst_last = sst[-1]
36 anomaly = sst_last - climato_same
37
38 # ----- plot the mean SST -----
39 fig = plt.figure(figsize=(11,4))
40 ax = fig.add_subplot(1, 1, 1, projection=ccrs.PlateCarree(central_longitude=180))
41 cf = ax.contourf(lon, lat, sst_mean, 30, cmap="jet",
42                 transform=ccrs.PlateCarree())
43 ax.coastlines()
44 plt.colorbar(cf, label="SST (deg C)", shrink=0.7)
45 plt.title("Mean SST over the record")
46
47 # ----- plot the anomaly -----
48 fig = plt.figure(figsize=(11,4))
49 ax = fig.add_subplot(1, 1, 1, projection=ccrs.PlateCarree(central_longitude=180))
50 cf = ax.contourf(lon, lat, anomaly, 30, cmap="RdBu_r",
51                 vmin=-3, vmax=3, transform=ccrs.PlateCarree())
52 ax.coastlines()
53 plt.colorbar(cf, label="SST anomaly (K)", shrink=0.7)
54 plt.title("SST anomaly for " + dtimes[-1].strftime("%Y-%m"))
55
56 plt.show()

```

Practice #11.1 – Nino3 index (Ch. 11)

```

1 # =====
2 # Practice #11.1 : Nino3 index and El Nino / La Nina years
3 # =====
4 # Approach:
5 # 1) Read sst.mnmean.nc.
6 # 2) Select the Nino3 box (5S-5N, 150W-90W) and area-average SST.
7 #    Careful with lon convention (0-360 vs -180..180).
8 # 3) Compute monthly climatology (1981-2010) and remove it -> anomaly.
9 # 4) Apply a 3-month running mean.
10 # 5) 5 consecutive months above +0.5 -> El Nino,
11 #    5 consecutive months below -0.5 -> La Nina.
12 # -----
13
14 import numpy as np
15 import matplotlib.pyplot as plt
16 import netCDF4 as nc
17

```

```

18 fh = nc.Dataset("sst.mnmean.nc", "r")
19 lat = fh.variables["lat"][:]
20 lon = fh.variables["lon"][:]
21 sst = fh.variables["sst"][:]
22 time = fh.variables["time"]
23 dtimes = nc.num2date(time[:], time.units)
24 fh.close()
25
26 # ----- select the Nino3 box (5S-5N, 150W-90W) -----
27 # lon in this file is 0-360; 150W = 210, 90W = 270
28 lat_mask = (lat >= -5) & (lat <= 5)
29 lon_mask = (lon >= 210) & (lon <= 270)
30
31 # subset the SST array with the two masks (two steps for clarity)
32 box = sst[:, lat_mask, :][:, :, lon_mask]
33
34 # area average (simple mean; box is narrow so cos(lat) hardly matters)
35 nino3 = np.mean(box, axis=(1, 2))
36
37 # ----- keep only 1981 onwards -----
38 years = np.array([d.year for d in dtimes])
39 months = np.array([d.month for d in dtimes])
40 mask = years >= 1981
41
42 nino3 = nino3[mask]
43 years = years[mask]
44 months = months[mask]
45
46 # ----- monthly climatology (1981-2010) -----
47 clim_mask = (years >= 1981) & (years <= 2010)
48 climatology = np.zeros(12)
49 for m in range(1, 13):
50     climatology[m-1] = np.mean(nino3[clim_mask & (months == m)])
51
52 # subtract climatology month by month to get the anomaly
53 anom = np.zeros_like(nino3)
54 for i in range(len(nino3)):
55     anom[i] = nino3[i] - climatology[months[i] - 1]
56
57 # ----- 3-month running mean -----
58 def running_mean(x, n=3):
59     """simple boxcar running mean of length n"""
60     out = np.full_like(x, np.nan)
61     for i in range(n-1, len(x)):
62         out[i] = np.mean(x[i-n+1:i+1])
63     return out
64
65 running = running_mean(anom, 3)
66
67 # ----- identify events -----
68 def find_events(series, threshold, sign="pos"):
69     events = []
70     count = 0
71     start = None
72     for i, val in enumerate(series):
73         if np.isnan(val):
74             count = 0
75             continue
76         if sign == "pos":
77             cond = val >= threshold
78         else:
79             cond = val <= -threshold
80         if cond:
81             if count == 0:
82                 start = i
83                 count += 1

```

```

84         else:
85             if count >= 5:
86                 events.append((start, i - 1))
87                 count = 0
88             if count >= 5:
89                 events.append((start, len(series) - 1))
90             return events
91
92 el_nino = find_events(running, 0.5, sign="pos")
93 la_nina = find_events(running, 0.5, sign="neg")
94
95 print("El Nino episodes:")
96 for s, e in el_nino:
97     print(" ", years[s], "-", months[s], " to ", years[e], "-", months[e])
98
99 print("La Nina episodes:")
100 for s, e in la_nina:
101     print(" ", years[s], "-", months[s], " to ", years[e], "-", months[e])
102
103 # ----- plot -----
104 t_axis = years + (months - 1) / 12.0
105 plt.figure(figsize=(11,4))
106 # plt.fill_between shades between two curves; nice for ENSO events
107 plt.fill_between(t_axis, running, 0.0,
108                  where=(running > 0.5), color="red", alpha=0.5,
109                  label="El Nino")
110 plt.fill_between(t_axis, running, 0.0,
111                  where=(running < -0.5), color="blue", alpha=0.5,
112                  label="La Nina")
113 plt.plot(t_axis, running, "k-", lw=1)
114 plt.axhline( 0.5, color="red", ls="--", lw=0.5)
115 plt.axhline(-0.5, color="blue", ls="--", lw=0.5)
116 plt.xlabel("Year")
117 plt.ylabel("Nino3 index (K)")
118 plt.title("Nino3 index and ENSO events, 1981-present")
119 plt.legend()
120 plt.grid(True)
121 plt.show()

```

Practice #11.2 – Quasi-Biennial Oscillation (Ch. 11)

```

1  # =====
2  # Practice #11.2 : QBO from the Singapore radiosonde data
3  # =====
4  # Approach:
5  #   The file Singapore.dat lists monthly zonal winds (unit 0.1 m/s)
6  #   at several pressure levels for many years.
7  #   1) Read it line by line and build a (time, level) array.
8  #   2) Plot the time - pressure diagram (a Hovmoller).
9  #   3) Alternating red / blue bands prove the QBO exists.
10 #   4) Pick one level (e.g. 30 hPa): sign changes about every ~28
11 #       months -> that is the QBO period.
12 # -----
13
14 import numpy as np
15 import matplotlib.pyplot as plt
16
17 # levels expected in the file (hPa)
18 levels = [10, 12, 15, 20, 25, 30, 35, 40, 45, 50, 60, 70, 80, 90, 100]
19
20 # We store winds in a dictionary keyed by (year, month)
21 records = {} # {(year, month): {level: wind}}
22
23 with open("Singapore.dat", "r") as f:
24     current_year = None
25     for line in f:

```

```

26     line = line.strip()
27     if not line:
28         continue
29     # a line with just a 4-digit number is the year header
30     if line.isdigit() and len(line) == 4:
31         current_year = int(line)
32         continue
33     parts = line.split()
34     # data lines look like: "10 -171 -282 ... -59"
35     try:
36         lev = int(parts[0])
37         values = [int(x) for x in parts[1:13]]
38     except (ValueError, IndexError):
39         continue # skip anything we do not understand
40     if lev not in levels:
41         continue
42     for m, v in enumerate(values, start=1):
43         key = (current_year, m)
44         if key not in records:
45             records[key] = {}
46         records[key][lev] = v * 0.1 # convert 0.1 m/s -> m/s
47
48 # turn the dictionary into a (time, level) array
49 sorted_keys = sorted(records.keys())
50 n_time = len(sorted_keys)
51 n_lev = len(levels)
52 U = np.full((n_time, n_lev), np.nan)
53 years = []
54 months = []
55 for i, (y, m) in enumerate(sorted_keys):
56     years.append(y)
57     months.append(m)
58     for j, lev in enumerate(levels):
59         if lev in records[(y, m)]:
60             U[i, j] = records[(y, m)][lev]
61
62 t_axis = np.array(years) + (np.array(months) - 1) / 12.0
63
64 # ----- Hovmoller -----
65 plt.figure(figsize=(11,5))
66 # contourf with a symmetric colour map so easterlies (negative) look
67 # different from westerlies (positive)
68 cf = plt.contourf(t_axis, levels, U.T, 21, cmap="RdBu_r",
69                  vmin=-30, vmax=30)
70 plt.gca().invert_yaxis() # low pressure = high altitude on top
71 plt.yscale("log") # pressure is usually shown log
72 plt.colorbar(cf, label="Zonal wind (m/s)")
73 plt.xlabel("Year")
74 plt.ylabel("Pressure (hPa)")
75 plt.title("Singapore zonal wind: QBO signature")
76
77 # ----- 1-D plot at one level to see the period -----
78 i30 = levels.index(30)
79 plt.figure(figsize=(11,3))
80 plt.plot(t_axis, U[:, i30], "b-")
81 plt.axhline(0, color="k", lw=0.5)
82 plt.xlabel("Year")
83 plt.ylabel("u at 30 hPa (m/s)")
84 plt.title("Zonal wind at 30 hPa - alternates every ~28 months")
85 plt.grid(True)
86 plt.show()
87
88 # ----- period estimation using zero crossings at 30 hPa -----
89 zero_cross = np.where(np.diff(np.sign(U[:, i30])))[0]
90 if len(zero_cross) > 1:
91     half_period = np.mean(np.diff(zero_cross))

```

```
92 print("Estimated QBO period ~", round(2 * half_period, 1), "months")
```

Practice #11.3 (bonus) – MJO from GPCP rainfall (Ch. 11)

```
1 # =====
2 # Practice #11.3 : MJO detection from GPCP daily rainfall
3 # =====
4 # Approach:
5 #   The Madden-Julian Oscillation shows up as a slow eastward
6 #   propagation of tropical precipitation.
7 #   1) Read the GPCP netCDF for 2 years.
8 #   2) Restrict to the tropics (10S - 10N), average over latitude
9 #       -> gives a (time, lon) array = a Hovmoller diagram.
10 #   3) Band-pass in time (20 - 90 days) to isolate the MJO signal;
11 #       we use a running-mean subtraction (very simple filter) since
12 #       we are not doing serious signal processing yet.
13 #   4) Show the Hovmoller : eastward-tilting stripes = MJO.
14 # -----
15
16 import numpy as np
17 import matplotlib.pyplot as plt
18 import xarray as xr
19
20 # example file path (user must download from the GPCP page)
21 fname = "gpcp_daily_2015_2016.nc"
22 ds = xr.open_dataset(fname)
23 pr = ds["precip"] # (time, lat, lon), units mm/day
24
25 # tropical strip
26 pr_trop = pr.sel(lat=slice(-10, 10))
27 # latitude average -> Hovmoller
28 pr_hov = pr_trop.mean(dim="lat") # (time, lon)
29
30 # very simple band pass: keep 21-90 day components
31 def running_mean_1d(x, n):
32     """1D running mean of length n; edges are just repeated"""
33     out = np.copy(x)
34     n2 = n // 2
35     for i in range(n2, len(x) - n2):
36         out[i] = np.mean(x[i-n2:i+n2+1])
37     return out
38
39 filtered = np.zeros_like(pr_hov.values)
40 for j in range(pr_hov.shape[1]):
41     series = pr_hov.values[:, j]
42     low_freq = running_mean_1d(series, 91) # removes >90 days
43     high_freq = running_mean_1d(series, 21) # keeps <=21 days
44     # keep the band we want: (smoothed 21-d) - (smoothed 91-d)
45     filtered[:, j] = high_freq - low_freq
46
47 # time axis in days from the start
48 time = np.arange(pr_hov.shape[0])
49 lon = pr_hov["lon"].values
50
51 plt.figure(figsize=(7,10))
52 # contourf with time going DOWN so eastward propagation looks tilted
53 cf = plt.contourf(lon, time, filtered, 21, cmap="RdBu_r")
54 plt.gca().invert_yaxis()
55 plt.colorbar(cf, label="Rainfall anomaly (mm/day)")
56 plt.xlabel("Longitude (deg)")
57 plt.ylabel("Day since start")
58 plt.title("MJO Hovmoller (tropical rainfall, 20-90 d band)")
59 plt.show()
60
61 # MJO features you should recognise:
62 # - eastward propagation (stripes tilt from lower-left to upper-right)
```

```

63 # - period between 30 and 60 days
64 # - strongest signal over the Indian Ocean / Maritime Continent

```

Practice #12 – Rainfall extremes with GEV (Ch. 12)

```

1  # =====
2  # Practice #12 : GEV analysis of Thai Binh rainfall (1961-2019)
3  # =====
4  # Approach:
5  # 1) Read the daily rainfall file (year, month, day, rainfall).
6  # 2) For each year, take the maximum daily rainfall -> Rx1day.
7  # 3) Fit a GEV distribution to the Rx1day series, plot the histogram
8  #    with the fitted density on top.
9  # 4) Return level for T = 5, 10, 20, 100 years:
10 #    xT = gev.ppf(1 - 1/T, shape, loc, scale)
11 # 5) Repeat for daily rainfall values >= 50 mm (heavy days).
12 # -----
13
14 import numpy as np
15 import matplotlib.pyplot as plt
16 from scipy.stats import genextreme as gev
17
18 fname = "R_THAIBINH_1961_2019.txt"
19
20 # 4 columns expected: year, month, day, R
21 data = np.loadtxt(fname)
22 year = data[:, 0].astype(int)
23 month = data[:, 1].astype(int)
24 day = data[:, 2].astype(int)
25 rain = data[:, 3]
26
27 # ----- step 1 : yearly maximum daily rainfall (Rx1day) -----
28 years_unique = np.unique(year)
29 Rx1day = []
30 for y in years_unique:
31     Rx1day.append(np.max(rain[year == y]))
32 Rx1day = np.array(Rx1day)
33
34 # ----- step 2 : histogram + GEV fit -----
35 shape, loc, scale = gev.fit(Rx1day)
36 print("GEV parameters for Rx1day: shape =", round(shape,3),
37       " loc =", round(loc,2), " scale =", round(scale,2))
38
39 x_grid = np.linspace(min(Rx1day), max(Rx1day)*1.3, 300)
40 pdf = gev.pdf(x_grid, shape, loc, scale)
41
42 plt.figure(figsize=(7,4))
43 # density=True normalises the histogram so it lines up with the PDF
44 plt.hist(Rx1day, bins=15, density=True, color="skyblue",
45         edgecolor="k", label="Rx1day histogram")
46 plt.plot(x_grid, pdf, "r-", lw=2, label="GEV fit")
47 plt.xlabel("Rainfall amount (mm/day)")
48 plt.ylabel("Frequency")
49 plt.title("Rx1day distribution, Thai Binh 1961-2019")
50 plt.legend()
51 plt.grid(True)
52
53 # ----- step 3 : return levels for Rx1day -----
54 return_periods = [5, 10, 20, 100]
55 print("Return levels for Rx1day :")
56 for T in return_periods:
57     xT = gev.ppf(1 - 1/T, shape, loc, scale)
58     print(" T =", T, "yr -> xT =", round(xT,1), "mm/day")
59
60 # ----- step 4 : same analysis for heavy days (>= 50 mm) -----
61 heavy = rain[rain >= 50.0]

```

```
62 shape2, loc2, scale2 = gev.fit(heavy)
63 print("GEV parameters for heavy days: shape =", round(shape2,3),
64       " loc =", round(loc2,2), " scale =", round(scale2,2))
65
66 x_grid2 = np.linspace(50, max(heavy)*1.2, 300)
67 pdf2     = gev.pdf(x_grid2, shape2, loc2, scale2)
68
69 plt.figure(figsize=(7,4))
70 plt.hist(heavy, bins=20, density=True, color="salmon",
71         edgecolor="k", label="R>=50 mm histogram")
72 plt.plot(x_grid2, pdf2, "b-", lw=2, label="GEV fit")
73 plt.xlabel("Rainfall amount (mm/day)")
74 plt.ylabel("Frequency")
75 plt.title("Distribution of heavy rainfall days (R>=50 mm)")
76 plt.legend()
77 plt.grid(True)
78
79 print("Return levels for heavy rainfall days :")
80 for T in return_periods:
81     xT = gev.ppf(1 - 1/T, shape2, loc2, scale2)
82     print("    T =", T, "yr -> xT =", round(xT,1), "mm/day")
83
84 plt.show()
```